



REGULAR EXPRESSION IN PYTHON

INTRODUCTION

- A **regular expression** is a **pattern** that describes a set of strings.
- It helps you check whether a string contains a specific pattern or extract and replace parts of it.
- **Importing Regex Module** : `import re`
- Common Uses of Regex

Purpose	Example
Validate	Email, phone number, password
Search	Find words, digits, dates in a text
Extract	Extract domain from URL, numbers from string
Replace	Replace whitespace, remove symbols

1. Character Classes (The "What")

Character classes tell the regex engine which specific types of characters you want to match.

Regex	Matches	Practical Example	Matches in Text
\d	Any single digit (0-9)	\d\d\d	415-555-1234
\D	Any character that is not a digit	\D	A1b2
\w	Any alphanumeric character + underscore	\w+	user_123
\W	Any character not alphanumeric (symbols, spaces)	\W	hello!
\s	Any whitespace (space, tab, newline)	\s	John Doe
\S	Any character that is not whitespace	\S+	Code123
.	Any character except a newline (\n)	c.t	cat, cut, c!t

1. Character Classes (The "What")

Custom Sets (Brackets [])

If the shortcuts above are too broad, you can build your own:

[aeiou]: Matches any single vowel.

[a-z]: Matches any single lowercase letter.

[A-Z0-9]: Matches any uppercase letter or number.

[^0-9]: The ^ inside brackets means **NOT**. This matches anything except a number.

2. Quantifiers (The "How Many")

Quantifiers are placed immediately *after* a character or character class to specify how many times that character must repeat.

Quantifier	Meaning	Practical Example	Matches in Text
*	0 or more times	apple*	appl (yes), apple (yes), applee (yes)
+	1 or more times	apple+	appl (No), apple (yes), applee (yes)
?	0 or 1 time (Optional)	apples?	apple or apples
{n}	Exactly n times	\d{5}	90210 (ZIP code)
{n,}	n or more times	\s{2,}	Matches 2 or more spaces in a row
{n,m}	Between n and m times	\w{3,5}	cat, math, words

Search, Match, Findall

Function	Where does it look?	What does it return?	Best used for...
re.match()	Only at the very beginning of the string.	A single Match object (or None).	Validating strict formats (like checking if an entire input is a valid ID).
re.search()	Scans the entire string, looking for the <i>first</i> match.	A single Match object (or None).	Finding if a pattern exists anywhere inside a sentence or log line.
re.findall()	Scans the entire string, looking for every match.	A list of strings (no match objects).	Extracting multiple pieces of data (like pulling all emails out of a document).

Search()

- The re.search() function is used to return the first occurrence of the pattern.
- This function searches the entire string and returns a match object if a match is found.
- Otherwise, it returns None in case the match is not found.

Example:

```
from re import *  
string = "Python is a general-purpose language"  
matched_string = search(r'general',string)  
print(matched_string)  
print(matched_string.start())  
print(matched_string.end())  
print(matched_string.span())
```

Compile()

- Converts a regular expression **string pattern** into a **compiled regex object** that can be reused for matching multiple times.

Without Compile()

```
re.search(pattern, text)
```

```
re.match(pattern, text)
```

```
re.findall(pattern, text)
```

With Compile()

```
regex = re.compile(pattern)
```

```
regex.search(text)
```

```
regex.match(text)
```

```
regex.findall(text)
```

- This improves:
 - ✓ **Readability**
 - ✓ **Performance** (faster if used repeatedly)

Split()

- used to **split a string based on a regex pattern**.
- split() splits a string **where the regular expression matches**, and returns a **list** of the resulting parts.
- Syntax : re.split(pattern, string, maxsplit=0)
- Example:

```
import re
```

```
text = "one-two-three-four"
```

```
result = re.split(r'-', text, maxsplit=2)
```

```
print(result)
```

Sub()

- sub() stands for **substitute**.
- Used to **search a pattern in a string and replace it with another string**.
- It replaces **all occurrences** of a regex pattern with a specified replacement.

Syntax : `re.sub(pattern, replacement, string, count=0)`

sub() is widely used for:

- ✓ cleaning text
- ✓ formatting strings
- ✓ removing unwanted characters
- ✓ replacing spaces, digits, symbols using regex

Subn()

- Used to **search a regex pattern, replace it with another string, and also return the number of replacements performed.**
- `re.subn()` works almost the same as `re.sub()`, but instead of returning only the modified string, it returns a **tuple**: (`new_string`, `number_of_replacements`)
- **Syntax** : `re.subn(pattern, replacement, string, count=0)`

Escape ()

- Used to **escape all special regex metacharacters in a string**, converting them into their **literal form**.
- `re.escape()` takes a string and **adds a backslash (\) before every regex metacharacter**, so the string can be safely used as a **literal pattern** in regex.

Why do we need `escape()`?

Regex contains special characters like: `. ^ $ * + ? { } [ ] ( ) | \`

If these appear in a string and we want them to be treated **as normal characters**, we need to escape them.

`re.escape()` does this **automatically**.

Syntax : `re.escape(string)`

MATCH OBJECTS

- **Match object** represents the result of a successful pattern match.
- They return a **Match object** containing information about **where and what** was matched.
- A Match object stores detailed information about the match, including:
 - The actual substring that matched the pattern
 - The start and end index positions
 - The original searched string
 - Any captured groups inside parentheses ()
- You can use Match object methods to extract this information.



END..